DBMS – CRUD Operations (Complete Notes)

INSERT | SELECT | UPDATE | DELETE

1. INSERT (Create)

Used to add new record(s) into a table.

Syntax:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Key Points:

- Specify column names (recommended).
- Number of values must match number of columns.
- Use single quotes for text values.
- You can insert multiple rows, use DEFAULT, use NULL, or insert using SELECT.

Examples:

`INSERT INTO Students (ID, Name, Age) VALUES (5, 'Karan', 21);`	Insert a single record (all columns).
`INSERT INTO Students (ID, Name) VALUES (11, 'Riya');`	Insert into selected columns only.
`INSERT INTO Students (ID, Name, Age) VALUES (13, 'Arjun', DEFAULT);`	Insert using default value (assumes default exists).
`INSERT INTO Students (ID, Name, Age) VALUES (15, 'Sahil', NULL);`	Insert a NULL value.
`INSERT INTO Students (ID, Name, Age) VALUES (8, 'Amit', 21), (9, 'Neha', 23);`	Insert multiple records in a single query.
`INSERT INTO Students (ID, Name, Age) SELECT ID, Name, Age FROM OldStudents;`	Insert using SELECT (copy from another table).

2. SELECT (Read)

Used to retrieve data from a table.

Syntax:

```
SELECT column1, column2, ...
FROM table_name
[WHERE condition]
[ORDER BY column_name]
[GROUP BY column_name]
[HAVING condition];
```

Key Points:

- * selects all columns.
- Use WHERE to filter records.
- You can use DISTINCT, ORDER BY, GROUP BY, HAVING, etc.

Examples:

`SELECT * FROM Students;`	Display all records and columns.
`SELECT Name, Age FROM Students;`	Display specific columns.
`SELECT DISTINCT City FROM Students;`	Display unique cities.
`SELECT * FROM Students WHERE Age > 20;`	Display students older than 20.
`SELECT * FROM Students ORDER BY Name ASC;`	Display all records sorted by name (ascending).

3. UPDATE (Update)

Used to modify existing record(s).

Syntax:

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Key Points:

- The WHERE clause is important to update specific records.
- You can update one or multiple columns.
- Values can be literals, expressions, or even from another table (using subquery).

Examples:

`UPDATE Students SET Age = 22 WHERE ID = 1;`	Update age for a specific student.
`UPDATE Students SET Name = 'Rahul', Age = 21 WHERE ID = 5;`	Update multiple columns.
`UPDATE Students SET Age = Age + 1 WHERE City = 'Mumbai';`	Update using an expression (increase age by 1 for all students from Mumbai).
`UPDATE Students SET Age = (SELECT Age FROM OldStudents WHERE OldStudents.ID = Students.ID) WHERE ID = 3;`	Update using a subquery (set age from another table).

4. DELETE (Delete)

Used to remove record(s) from a table.

Syntax:

```
DELETE FROM table_name
WHERE condition;
```

Key Points:

- The WHERE clause is important to delete specific records.
- Without WHERE, all records in the table will be deleted.
- You can delete multiple records based on a condition.

Examples:

`DELETE FROM Students WHERE ID = 5;`	Delete a specific student.
`DELETE FROM Students WHERE Age < 18;`	Delete all students younger than 18.
`DELETE FROM Students WHERE City = 'Delhi';`	Delete all students from Delhi.

DBMS – JOINS (Complete Notes)

Types, Syntax, Examples, and When to Use

What is a JOIN?

- A JOIN is used to combine rows from two or more tables based on a related column (usually a key).
- It allows you to retrieve data from multiple tables in a single query.
- Without JOINs, you can only get data from one table at a time.

Sample Tables (used in examples)

Students

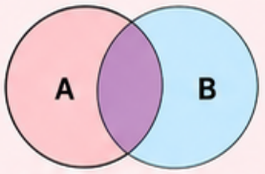
ID	Name	DeptID
1	Amit	10
2	Neha	20
3	Rahul	10
4	Priya	NULL

Departments

DeptID	DeptName
10	Computer Science
20	Information Technology
30	Cyber Security

1. INNER JOIN

- Returns only the matching records from both tables.
- Rows that do not have a match in either table are excluded.



Only matching records

Syntax:

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

Example: Get students with their department names.

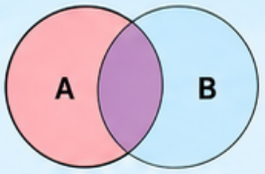
```
SELECT s.ID, s.Name, d.DeptName
FROM Students s
INNER JOIN Departments d
ON s.DeptID = d.DeptID;
```

Result:

ID	Name	DeptName
1	Amit	Computer Science
2	Neha	Information Technology
3	Rahul	Computer Science

2. LEFT JOIN (LEFT OUTER JOIN)

- Returns all records from the left table and matching records from the right table.
- If there is no match, NULL values are returned for the right table columns.



All from left table + matching from right

Syntax:

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.column = table2.column;
```

Example: Get all students and their department names (even if the department is not assigned).

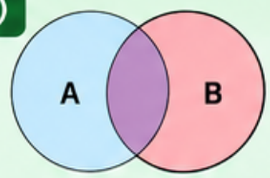
```
SELECT s.ID, s.Name, d.DeptName
FROM Students s
LEFT JOIN Departments d
ON s.DeptID = d.DeptID;
```

Result:

ID	Name	DeptName
1	Amit	Computer Science
2	Neha	Information Technology
3	Rahul	Computer Science
4	Priya	NULL

3. RIGHT JOIN (RIGHT OUTER JOIN)

- Returns all records from the right table and matching records from the left table.
- If there is no match, NULL values are returned for the left table columns.



All from right table + matching from left

Syntax:

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.column = table2.column;
```

Example: Get all departments and their students (even if no student is assigned).

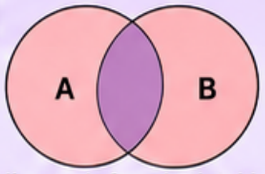
```
SELECT s.ID, s.Name, d.DeptName
FROM Students s
RIGHT JOIN Departments d
ON s.DeptID = d.DeptID;
```

Result:

ID	Name	DeptName
1	Amit	Computer Science
3	Rahul	Computer Science
2	Neha	Information Technology
NULL	NULL	Cyber Security

4. FULL JOIN (FULL OUTER JOIN)

- Returns all records when there is a match in either table.
- If there is no match, NULL values are returned for the missing side.



All records from both tables (matching + non-matching)

Syntax:

```
SELECT columns
FROM table1
FULL JOIN table2
ON table1.column = table2.column;
```

Example: Get all students and all departments.

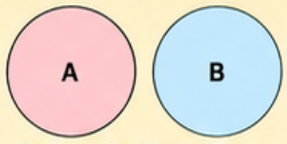
```
SELECT s.ID, s.Name, d.DeptName
FROM Students s
FULL JOIN Departments d
ON s.DeptID = d.DeptID;
```

Result:

ID	Name	DeptName
1	Amit	Computer Science
2	Neha	Information Technology
3	Rahul	Computer Science
4	Priya	NULL
NULL	NULL	Cyber Security

5. CROSS JOIN

- Returns the Cartesian product (all possible combinations) of rows from both tables.
- Every row from the first table is combined with every row from the second table.



All combinations of rows

Syntax:

```
SELECT columns
FROM table1
CROSS JOIN table2;
```

Example: Combine every student with every department.

```
SELECT s.Name, d.DeptName
FROM Students s
CROSS JOIN Departments d;
```

Result:

Name	DeptName
Amit	Computer Science
Amit	Information Technology
Amit	Cyber Security
Neha	Computer Science

... (all combinations) ...

6. SELF JOIN

- A table is joined with itself.
- Useful when you need to compare rows within the same table (e.g., employee and manager).

Employees

ID	Name	ManagerID
1	Raj	NULL
2	Amit	1
3	Neha	1
4	Karan	2

Syntax:

```
SELECT a.column, b.column
FROM table_name a
JOIN table_name b
ON a.column = b.column;
```

Example: Get each employee and their manager name.

```
SELECT e.Name AS Employee,
       m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m
ON e.ManagerID = m.ID;
```

Syntax:

Employee	Manager
Raj	NULL
Amit	Raj
Neha	Raj
Karan	Amit

DBMS – WHERE Clause (Complete Notes)

Syntax | Operators | Examples | Common Conditions | Multiple Conditions | Special Operators

1. WHERE Clause – Basics

- Used to filter records based on a condition.
- Only rows that satisfy the condition are returned.
- Used with SELECT statement.

Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example: Get students with age 21.

```
SELECT *
FROM Students
WHERE Age = 21;
```

Result (sample):

ID	Name	Age	DeptID
3	Rahul	21	10

2. Comparison Operators

- Used to compare values in the WHERE clause.
- Common operators: =, <>, !=, >, <, >=, <=

Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE column_name operator value;
```

Examples:

Operator	Example Query	Meaning
=	`SELECT * FROM Students WHERE Age = 21;`	Equal to
<> or !=	`SELECT * FROM Students WHERE Age <> 21;`	Not equal to
>	`SELECT * FROM Students WHERE Age > 20;`	Greater than
<	`SELECT * FROM Students WHERE Age < 20;`	Less than
>=	`SELECT * FROM Students WHERE Age >= 18;`	Greater than or equal to
<=	`SELECT * FROM Students WHERE Age <= 22;`	Less than or equal to

3. Logical Operators (AND, OR, NOT)

- Used to combine multiple conditions.
- **AND**: both conditions must be true.
- **OR**: at least one condition must be true.
- **NOT**: negates the condition.

Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 AND/OR/NOT condition2;
```

Examples:

```
SELECT * FROM Students
WHERE Age > 18 AND DeptID = 10;
```

ID	Name	Age	DeptID
3	Rahul	21	10

```
SELECT * FROM Students
WHERE DeptID = 10 OR DeptID = 20;
```

ID	Name	Age	DeptID
1	Amit	20	10
2	Neha	19	20

```
SELECT * FROM Students
WHERE NOT DeptID = 10;
```

ID	Name	Age	DeptID
2	Neha	19	20
4	Priya	22	30

4. BETWEEN Operator

- Used to select values within a range (inclusive).

Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

Example: Students with age between 18 and 22.

```
SELECT * FROM Students
WHERE Age BETWEEN 18 AND 22;
```

Result (sample):

ID	Name	Age	DeptID
1	Amit	20	10
2	Neha	19	20

5. IN Operator

- Used to match any value in a list.

Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE column_name IN (value1, value2, ...);
```

Example: Students from department 10 or 30.

```
SELECT * FROM Students
WHERE DeptID IN (10, 30);
```

Result (sample):

ID	Name	Age	DeptID
1	Amit	20	10
4	Priya	22	30

6. LIKE Operator (Pattern Matching)

- Used to search for text patterns.
- % : any number of characters
- _ : single character

Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE column_name LIKE pattern;
```

Examples:

```
SELECT * FROM Students
WHERE Name LIKE 'A%';
```

ID	Name	Age	DeptID
1	Amit	20	10

```
SELECT * FROM Students
WHERE Name LIKE '%a';
```

ID	Name	Age	DeptID
2	Neha	19	20
4	Priya	22	30

7. IS NULL / IS NOT NULL

- Used to check for NULL values.

Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE column_name IS NULL;
-- or
WHERE column_name IS NOT NULL;
```

Examples:

```
SELECT * FROM Students
WHERE DeptID IS NULL;
```

ID	Name	Age	DeptID
4	Priya	22	NULL

```
SELECT * FROM Students
WHERE DeptID IS NOT NULL;
```

ID	Name	Age	DeptID
1	Amit	20	10
2	Neha	19	20

8. ORDER BY with WHERE

- Used to sort the filtered results.

Syntax:

```
SELECT column1, column2, ...
FROM table_name
WHERE condition
ORDER BY column_name ASC|DESC;
```

Example: Students older than 18, sorted by age (descending).

```
SELECT * FROM Students
WHERE Age > 18
ORDER BY Age DESC;
```

ID	Name	Age	DeptID
4	Priya	22	30
3	Rahul	21	10
1	Amit	20	10